

Laboratory 3: Serial Communication

Objectives

1. Explain the purpose of serial communication in embedded systems.
2. Configure serial communication using `Serial.begin()`.
3. Send data to the Serial Monitor using `Serial.print()` and `Serial.println()`.
4. Read user input from the Serial Monitor using `Serial.read()` and `Serial.readString()`.
5. Explain how characters are represented using ASCII codes.
6. Use `if-else` and `switch-case` statements to respond to serial input.
7. Develop a simple interactive program using the Serial Monitor and the M5Stack LCD.

Key Concepts:

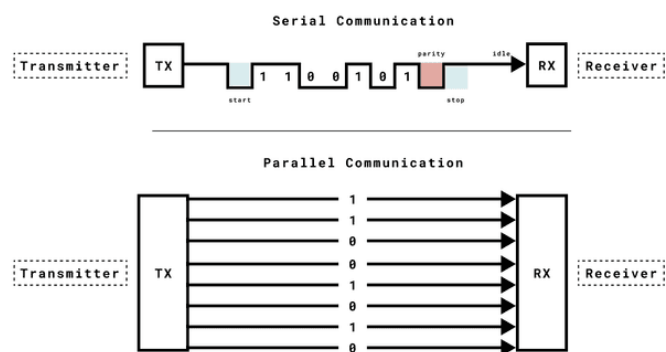
1. What is Serial Communication?

Serial communication is a method for using a sequence of bits to transfer a byte of data between two devices. In the digital system, we use 8 bits to represent a **byte** of data. A byte is generally equal to a character.

With a serial communication, the data transfer between two devices can be done with just two wires (one for sending and the other one for receiving). This kind of communication is cheaper comparing to parallel communication where 8 wires are needed to transmit a byte of data.

Because there is no shared clock signal, both devices must agree on the speed of transmission in advance. This speed is called the baud rate (bits per second, bps). If the sender and receiver use different baud rates, the data will be interpreted incorrectly.

In M5Stack, serial communication uses **UART** (Universal Asynchronous Receiver-Transmitter). The most common baud rate used in the course is **115200**.



2. Serial Functions in Arduino

The following functions are used for serial communication:

Function	Description
<code>Serial.begin(speed)</code>	Set the baud rate. Must be called in <code>setup()</code> .
<code>Serial.print(data)</code>	Send data to Serial Monitor (no new line).
<code>Serial.println(data)</code>	Send data followed by a new line character.
<code>Serial.available()</code>	Returns the number of bytes waiting to be read. Use this to check if any input has arrived.
<code>Serial.read()</code>	Reads one byte (character) from the serial buffer.
<code>Serial.readString()</code>	Reads all waiting bytes as a String (until timeout).

3. Serial Buffer

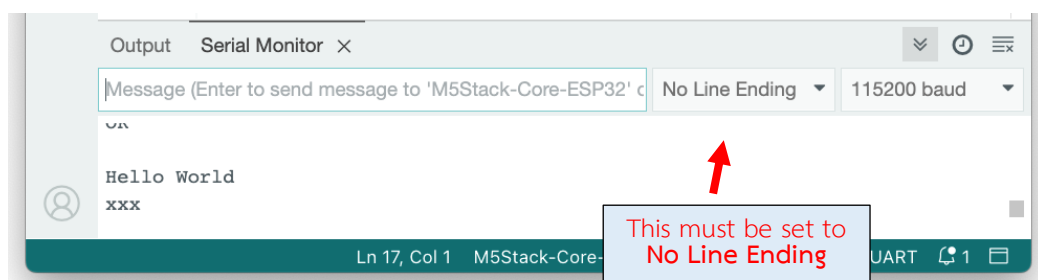
Think of the serial buffer as a mailbox. When you type characters in the Serial Monitor and press Enter, all the characters are placed into the mailbox – one letter per slot. The program then picks them up one at a time using `Serial.read()`.

For example, if you type "abc" and press **Enter**:

- The mailbox receives 3 letters: 'a', 'b', 'c' (plus a line-ending character).
- `Serial.available()` returns 3 (or more).
- Each call to `Serial.read()` picks up one letter.
- `loop()` runs multiple times until the mailbox is empty.

Always check the mailbox before reading from it. If you call `Serial.read()` when the buffer is empty, it returns -1, which is not a valid character and will cause unexpected behavior:

```
void loop() {  
  if (Serial.available() > 0) { // check first – is there anything to read?  
    int b = Serial.read();      // only then pick up one letter  
    Serial.print('x');  
  }  
}
```



The figure above shows the result of the previous sketch when you type in any three characters in the input box of the serial monitor. The number of x's reflects the number of characters you type in.

Line Ending Characters

When you press Enter in the Serial Monitor, it does not just signal **"done"** – it also sends one or two special characters after your text. These are called line ending characters.

Character	Meaning
\n (ASCII 10)	New Line – moves to the next line
\r (ASCII 13)	Carriage Return – moves cursor to start of line

The Serial Monitor has a dropdown menu to control which line ending is appended to your input. Choosing **"No Line Ending"** sends only your typed characters. The other options add '\n', '\r', or both. This matters when using `Serial.read()`, because those extra characters will appear as separate byte in the buffer.

4. ASCII

ASCII (American Standard Code for Information Interchange) is a standard that assigns a number to each character. For example, the character 'A' has ASCII value 65, and 'B' has value 66. Digits follow the same pattern: '0' = 48, '1' = 49, and so on.

When `Serial.read()` receives a character, it returns the ASCII integer value, not the character itself. To convert it back to a printable character, use a `(char)` cast:

```
int ascii = Serial.read();  
char c = (char) ascii;  
  
or  
  
char c = (char) Serial.read();
```

5. Useful Operators

You already know the basic arithmetic operators: +, -, *, /. This lab introduces two more:

Increment and Decrement

A shorthand for adding or subtracting 1 from a variable:

```
count++; // same as count = count + 1  
count--; // same as count = count - 1
```

Modulo – remainder after division (%)

Returns the remainder when one number is divided by another:

```
7 % 2 → 1    // 7 divided by 2, remainder is 1  
8 % 2 → 0    // 8 divided by 2, remainder is 0
```

A common use is to check whether a number is odd or even:

```
if (count % 2 != 0) {  
    // count is odd  
}
```

6. Arduino Control Structures (Decisions) and operators

Arduino uses the same control structures as C/C++. You have already used `if` in Lab 2. Here are the additional ones used in this lab:

if-else

Choose between two or more actions depending on a condition:

```
if (count % 2 != 0) {  
    // count is odd  
} else {  
    // count is even  
}
```

switch-case

When you need to choose between several specific values, **switch-case** is cleaner than many **if-else** statements:

```
switch (input) { // input is a character or an integer  
    case 'r':  
        // runs when input == 'r'  
        break; // exit the switch block  
    case 'g':  
        // runs when input == 'g'  
        break;  
    default:  
        // runs when no case matches  
        break;  
}
```

Important: Without **break**, execution continues into the next case automatically.

7. String Type

A **String** stores a sequence of characters as a single value. Unlike a plain **char** (one character), a **String** can hold a whole word or sentence.

```
String text = "Hello M5Stack"; // stores words Hello M5Stack  
int len = text.length(); // 13
```

Serial.readString() reads all characters currently waiting in the serial buffer and returns them as a single **String**. This is convenient when you want to receive a whole word at once, rather than reading character by character.

Lab Exercises

Save each task as a separate sketch: **lab_3_1.ino**, **lab_3_2.ino**, ..., **lab_3_6.ino**.

Task 1: Baud Rate and Serial Monitor (lab_3_1)

Objective: In this task, you will observe what happens when the baud rate on the M5Stack and the Serial Monitor do not match.

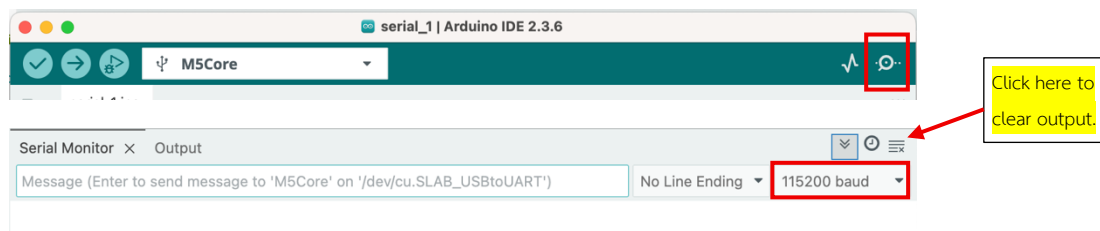
1.1 Create New Sketch and copy the following code and save as **lab_3_1**.

```
#include <M5Unified.h>

void setup() {
  M5.begin();
  Serial.begin(57600); // M5Stack sends at 57600 baud
  Serial.println("Lab 3 - Task 1");
}

void loop() {
  Serial.print("Can you read?");
}
```

1.2 Upload the sketch. Open the Serial Monitor. Make sure the baud rate drop down is set to **115200**. Observe the output – you should see unreadable characters.



1.3 Change the Serial Monitor dropdown to other values (9600, 57600, 115200) until the text becomes readable. Record which value works.

1.4 **TODO:** Modify the sketch to:

- Change `Serial.begin(57600)` to `Serial.begin(115200)` so the default Serial Monitor setting works.
- Change `Serial.print` to `Serial.println` so "Can you read?" appears on a new line each time.

Task 2: Button State → Serial Output (lab_3_2)

Objective: In Lab 2, you used the LCD to show button states. In this task, you will use the Serial Monitor as an event log. Along the way, you will also discover why `delay()` can cause problems in `loop()` and how to fix it properly.

2.1 Create a new sketch and copy the code below. Save as **lab_3_2**.

```
#include <M5Unified.h>

int count = 0; // count total button presses

void setup() {
  M5.begin();
  Serial.begin(115200);
  Serial.println("Lab 3 - Task 2");
  M5.Display.setTextSize(3);
}

void loop() {
  if (M5.BtnA.isPressed()) {
    count++;
    Serial.print("[");
    Serial.print(count);
    Serial.println("] Button A--");
  }
  if (M5.BtnB.isPressed()) {
    count++;
    Serial.print("[");
    Serial.print(count);
    Serial.println("] Button --B");
  }
}
```

2.2 Upload. Press and hold Button A. Observed the Serial Monitor – the output floods very fast because `loop()` runs thousands of times per second. Also notice that nothing appears at all – find the missing statement and add it.

2.3 Add `delay(500)` at the end of `loop()` to slow things down. Upload and test:

- Press and hold Button A – output appears every 0.5 seconds.
- Now press and hold Button A, then **also press Button B at the same time** – observe whether Button B is always detected.

You will notice that Button B event are sometime missed. This is because `delay()` pauses the entire program for the specified time — while M5Stack is waiting, it cannot do anything else, including reading button input.

2.4 Remove `delay()` and change both `isPressed()` to `wasPressed()`. Upload and test:

- Press and hold Button A – output appears only once per press, no flood.
- Press Button A and Button B quickly – both are always detected.

Note: `wasPressed()` fires only once per press and never blocks `loop()`. This is the correct way to handle button input – avoid using `delay()` in `loop()` when you need to respond to multiple inputs.

2.5 **TODO:** Now add the event log feature. Update the sketch so that:

- Button A logs "[count] Button A pressed" to the Serial Monitor and displays the letter on the LCD. (no need to clear the screen first)
- Button B logs "[count] Button B pressed" to the Serial Monitor and displays the letter on the LCD. (no need to clear the screen first)
- Button C logs "[count] Screen cleared" to the Serial Monitor and clears the LCD. (Hint: after clearing, reset the cursor to the top-left corner so the next letter appears in the correct position.)

Note: Clearing the LCD does not erase the Serial Monitor history. This is the key difference between the two output channels.

Task 3: Reading One Byte --- **Serial.read()** (lab_3_3)

Objective: Explore how `Serial.read()` receives data one byte at a time, and how ASCII codes relates to characters.

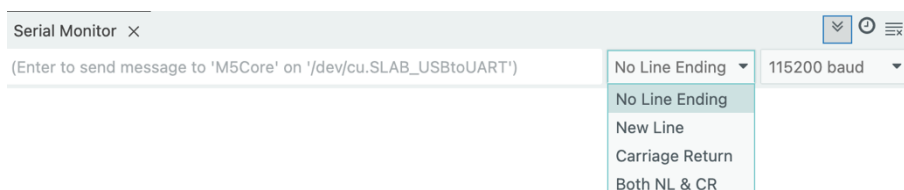
3.1 Create a new sketch and copy the code below and save as **lab_3_3**:

```
#include <M5Unified.h>

void setup() {
  M5.begin();
  M5.Display.setTextSize(3);
  M5.Display.setTextColor(TFT_BLUE, TFT_YELLOW);
  Serial.begin(115200);
  Serial.println("Lab 3 - Task 3");
}

void loop() {
  M5.update();
  if (Serial.available() > 0) {    // check if data is waiting
    int readByte = Serial.read(); // read one byte
    Serial.print('[');
    Serial.print(readByte);        // print ASCII value (integer)
    Serial.print(' ');
    Serial.print(readByte, HEX);    // print ASCII value in HEX
    Serial.println(']');
  }
}
```

3.2 Make sure the Serial Monitor line ending is set to **No Line Ending**. Upload and type a single character such as 'A' or '2'. Observe the output --- you should see the ASCII value in decimal and hexadecimal.

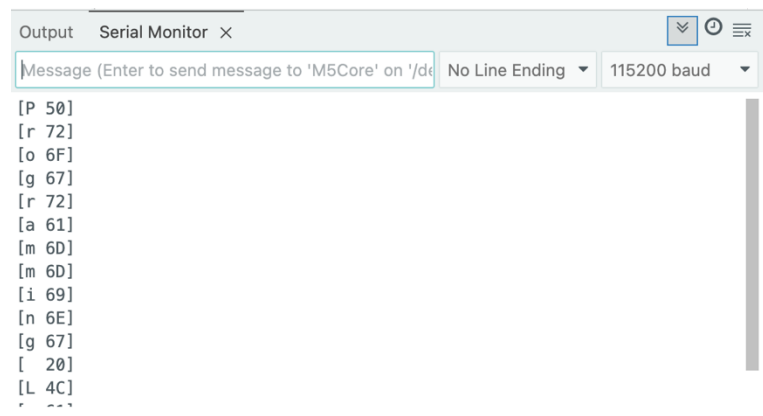


- 3.3 Change the line ending dropdown to **New Line**, then **Carriage Return**, then **Both NL & CR**. Type a character each time and observe the extra output lines. What are the extra ASCII value?
- 3.4 To see the actual character instead of its ASCII number, replace `Serial.print(readByte)` with `Serial.print((char)readByte)`. Re-upload and test.
- 3.5 Set line ending back to **No Line Ending**. Type "abc" and send. Observe that you get 3 separate output lines --- one for each character. This shows how the serial buffer works: all 3 characters wait in the buffer, and `Serial.read()` picks them up one by one across multiple loop executions.
- 3.6 **TODO:** Modify the sketch so that each received character is displayed in the following format:
- ```
[A 65]
[B 66]
[2 50]
```

The output should show in the Serial Monitor: the **actual character** on the left and its ASCII **decimal value** on the right, separated by a space.

Set the line ending to **No Line Ending** and type several characters to test.

The pictures below show output when type "Programming Lab".



#### Task 4: Reading Full Text --- `Serial.readString()` + `if-else` (lab\_3\_4)

**Objective:** Use `Serial.readString()` to receive a whole word at once, then use `if-else` to control what is displayed.

- 4.1 Create a new sketch and copy the code below:

```
#include <M5Unified.h>

int count = 0; // counts how many strings have been received

void setup() {
 M5.begin();
 Serial.begin(115200);
 Serial.println("Lab 3 - Task 4");
 M5.Display.setTextSize(3);
}
```

```
}

void loop() {
 M5.update();
 if (Serial.available() > 0) {
 String text = Serial.readString(); // read the whole input
 count++; // increment counter
 Serial.print(count);
 Serial.print(" - ");
 Serial.println(text); // show in Serial Monitor
 M5.Display.println(text); // show on LCD
 }
}
```

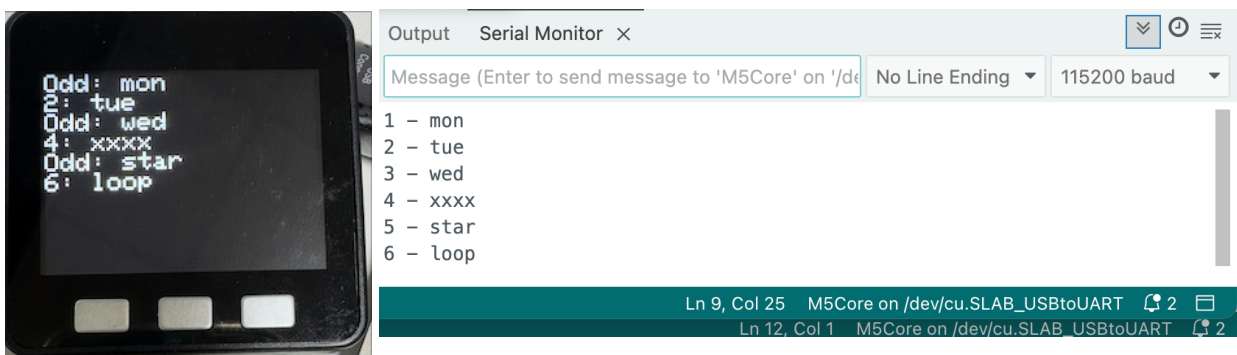
4.2 Upload and test by typing several words: "mon", "tue", "wed", "xxxx", "star", "loop". Observe the counter and the LCD output.

4.3 **TODO:** Modify the sketch using `if-else` so that the LCD display:

- When count is **odd**, display "Odd: " followed by the text.
- When count is **even**, display the count value followed by ": " followed by the text.

**Hint:** Use the modulo operator `%` to check odd/even. Refer to Key Concepts section 6 if needed.

The picture shows LCD display when six texts are input. ("mon", "tue", "wed", "xxxx", "star", "loop")



## Task 5: Color Control with **switch-case** (lab\_3\_5)

**Objective:** Use `switch-case` to respond to a single character input and change the LCD text color accordingly.

5.1 Create a new sketch using the skeleton code below:

```
#include <M5Unified.h>

void setup() {
 M5.begin();
 Serial.begin(115200);
 Serial.println("Lab 3 - Task 5");
 M5.Display.setTextSize(3);
}
```

```
void loop() {
 M5.update();
 if (Serial.available() > 0) {
 char input = (char)Serial.read(); // read one character
 // TODO: print input to Serial Monitor (one line per character)
 switch (input) {
 case 'r':
 case 'R': // both 'r' and 'R' reach this
 M5.Display.setTextColor(TFT_RED, TFT_BLACK);
 M5.Display.println("RED");
 break;
 // TODO: add cases for g/G and b/B
 default:
 // TODO: clear screen and show RESET in TFT_YELLOW
 break;
 }
 }
}
```

Upload and test by typing: r, g, b, R, G, B, then any other character (e.g. x, 9).

### Task 6: Answer the questions

### Task 7: Reflection assignment in myCourseVille

**Lab 3: Serial Communication**

Section: \_\_\_\_\_ Date: \_\_\_\_\_

**Members**

Name: \_\_\_\_\_ Student ID: \_\_\_\_\_

Name: \_\_\_\_\_ Student ID: \_\_\_\_\_

Name: \_\_\_\_\_ Student ID: \_\_\_\_\_

| Task | Description                                                                                                                       | Graded by |
|------|-----------------------------------------------------------------------------------------------------------------------------------|-----------|
| 1    | Baud rate mismatch --- change Serial Monitor and fix baud rate in code so text is readable.                                       |           |
| 2    | Serial as event log: Button A/B logs to Serial with counter, Button C clears LCD but not Serial history.                          |           |
| 3    | <code>Serial.read()</code> : observe ASCII values, line endings, and buffer behavior.<br>Serial log [{letter} {ascii in decimal}] |           |
| 4    | <code>Serial.readString()</code> + if-else to control LCD<br>Odd → Odd: + text<br>Even → value: + text                            |           |
| 5    | switch-case: control LCD text color ('rgbRGBx')                                                                                   |           |

Answer questions in next page

## 6. Answer the following questions.

Q1. In Task 1, why did the Serial Monitor show unreadable characters at first? What did you need to change to make the text readable?

Q2. In Task 2, what was the missing statement, and why is it required every loop?

Q3. In task 2.3, why were Button B events sometimes missed when `delay(500)` was in `loop()`? What does this tell you about using `delay()` in a program that needs to respond to multiple inputs?

Q4. After pressing A, B, A, B then clearing LCD with Button C: what did the LCD show? What did the Serial Monitor show? What can the Serial Monitor do that the LCD cannot?

Q5. In Task 3.5, you typed "abc" with line ending set to **No Line Ending** and saw 3 separate output lines. Why does one input produce 3 lines? Use the concept of the serial buffer in your explanation.

Q6. What is the difference between `Serial.read()` and `Serial.readString()`? Give an example situation where you should prefer each one.

## 7. Reflection about this lab. See assignment in MyCourseVille.